TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MINOR PROJECT REPORT

ON

HUMAN PRESENCE DETECTION USING DISTRIBUTED INFERENCE ON RESOURCE CONSTRAINED EDGE DEVICES

Submitted By:

Ishwor Raj Pokhrel	(THA079BEI013)
Pramit Khatri	(THA079BEI020)
Preeti Rijal	(THA079BEI028)
Sampada Ghimire	(THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Asst. Prof. Suwarna Lingden

March , 2026

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MINOR PROJECT REPORT
ON
HUMAN PRESENCE DETECTION USING DISTRIBUTED INFERENCE ON RESOURCE CONSTRAINED EDGE DEVICES

Submitted By:

Ishwor Raj Pokhrel	(THA079BEI013)
Pramit Khatri	(THA079BEI020)
Preeti Rijal	(THA079BEI028)
Sampada Ghimire	(THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

March , 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a minor project work entitled **"Human Presence Detection Using Distributed Inference On Resource Constrained Edge Devices"** submitted by **Ishwor Raj Pokhrel, Pramit Khatri , Preeti Rijal , Sampada Ghimire** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's Degree of Electronics, Communication, and Information Engineering.

Project Supervisor
Asst. Prof. Suwarna Lingden
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

External Examiner

Head of Department
Asst. Prof. Umesh Kanta Ghimire
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

March , 2026

DECLARATION

We hereby declare that the project entitled, **"Human Presence Detection Using Distributed Inference On Resource Constrained Edge Devices"** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Electronics, Communication and Information Engineering** is a bonafide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Ishwor Raj Pokhrel	(THA079BEI013)
Pramit Khatri	(THA079BEI020)
Preeti Rijal	(THA079BEI028)
Sampada Ghimire	(THA079BEI036)

Date: March , 2026

COPYRIGHT

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project supervisor, **Asst. Prof. Suwarna Lingden**, for his guidance, invaluable feedback, and constant support for the project. His expertise, experience in domain knowledge of Scheduling and encouragement have been pivotal in shaping this project.

We are also deeply thankful to **Asst. Prof. Umesh Kanta Ghimire**, Head of the Department, and the entire Department of Electronics and Computer Engineering for providing the necessary facilities and a supportive academic environment for our study.

Finally, we extend our sincere thanks to all the teachers, friends, and both direct and indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Ishwor Raj Pokhrel	(THA079BEI013)
Pramit Khatri	(THA079BEI020)
Preeti Rijal	(THA079BEI028)
Sampada Ghimire	(THA079BEI036)

ABSTRACT

150-300 words, include: Problem context, Approach, Key results, Impact. Avoid citations, unexpanded acronyms, detailed implementation.

Keywords—Combinatorial optimization, constraint satisfaction, genetic algorithm, hyper-heuristics, metaheuristic search, NP-hard problems, reinforcement learning, university course timetabling problem.

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL . . . . . **i**

DECLARATION . . . . . **ii**

COPYRIGHT . . . . . **iii**

ACKNOWLEDGEMENT . . . . . **iv**

ABSTRACT . . . . . **v**

LIST OF FIGURES . . . . . **viii**

LIST OF TABLES . . . . . **ix**

LIST OF ABBREVIATIONS . . . . . **ix**

1. Introduction **1**
- 1.1. Background 1
- 1.2. Motivation 2
- 1.3. Problem Definition 3
- 1.4. Objectives 4
- 1.5. Scope and Applications 4

2. LITERATURE REVIEW **5**

3. REQUIREMENTS ANALYSIS **7**
- 3.1. Hardware Requirements 7
- 3.2. Software & Framework Requirements 7
- 3.3. Non-Functional Requirements 7

4. SYSTEM ARCHITECTURE AND METHODOLOGY **9**
- 4.1. Hardware Used 9
- 4.2. Hardware Specifications and system topology 12
- 4.3. Model Selection and Optimization Pipeline 17
- 4.4. Evaluation Metrics and Performance Framework 26

5. RESULTS AND ANALYSIS **28**
- 5.1. LBCNN Architecture Performance 28
- 5.2. Test-Set Evaluation 29

6. REMAINING TASKS **31**

7. CONCLUSION **32**
7.1. Achievements 32
7.2. Limitations 32
7.3. Future Work 32
7.4. Final Remarks 32

APPENDICES **33**

A. PROJECT BUDGET **33**

B. PROJECT TIMELINE **34**
B.1. Gantt Chart 34

C. IMPLEMEMNTED CODE **35**

REFERENCES **36**

LIST OF FIGURES

Figure 4-1. Wiring Interface between Programming (FTDI), Processing (ESP32-S3), and Visualization (I2C LCD) Units. 9
Figure 4-2. System Architecture showing main components and their interactions. 12
Figure 4-3. Detailed Data Flow Diagram of the Tiling and Inference Pipeline. . . 13
Figure 4-4. RGB to Grayscale 14
Figure 4-5. Proposed Multi-Tile Attention-Based CNN Architecture 17
Figure 4-6. Detailed Schematic of the Local Binary Convolution Module 18
Figure 5-1. Training and validation loss over 45 epochs. 28
Figure 5-2. Normalised confusion matrix on the test set. 30

LIST OF TABLES

Table 4-1. Wiring the FTDI to ESP32-CAM 10
Table 4-2. Wiring the I2C LCD to ESP32 11
Table 5-1. Training Run Summary 29
Table 5-2. Per-Class Classification Report on the Test Set (n = 92) 29
Table A-1. Budget Estimation 33
Table B-1. Gantt Chart for Project Timeline 34

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area Under the Curve
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
COCO	Common Objects in Context
DAT	Distributed Adaptive Tiling
ERF	Effective Receptive Field
ESP	Espressif Systems
ESP-DL	Espressif Deep Learning library
ESP-NOW	Espressif Non-Beaconing Wireless
FLOPs	Floating Point Operations
FN	False Negative
FP	False Positive
FP32	32-bit Floating Point
FPS	Frames Per Second
GAP	Global Average Pooling
GPU	Graphics Processing Unit
GSD	Ground Sampling Distance
HPD	Human Presence Detection
HVAC	Heating, Ventilation, and Air Conditioning
INT4	4-bit Integer
INT8	8-bit Integer
IoT	Internet of Things
IoU	Intersection over Union
IR	Infrared
KB	Kilobyte

LBC	Local Binary Convolutions
LBCNN	Local Binary Convolutional Neural Network
LBP	Local Binary Patterns
LMMs	Large Multimodal Models
mac	Multiply-Accumulate Operations
mAP	Mean Average Precision
MB	Megabyte
MCU	Microcontroller Unit
ms	Milliseconds
OD	Object Detection
OOM	Out of Memory
PCs	Personal Computers
PIR	Passive Infrared
PTQ	Post-Training Quantization
Q-CNN	Quantized Convolutional Neural Network
QAT	Quantization-Aware Training
R-CNN	Regions with Convolutional Neural Network
SAHI	Slicing Aided Hyper Inference
SL	Split Learning
SOD	Small Object Detection
SR	Squeeze Ratio
SRAM	Static Random-Access Memory
TFML	TensorFlow Lite for Microcontrollers
TinyML	Tiny Machine Learning
TP	True Positive
UAV	Unmanned Aerial Vehicle
UHD	Ultra-high Definition

WSN	Wireless Sensor Network
YOLO	You Only Look Once

1. INTRODUCTION

While Convolutional Neural Networks (CNNs) have mastered object detection in controlled environments, moving to real-world, high-resolution processing is still hampered by significant hardware constraints. In many scenarios, simply shrinking a high-definition image to fit into a small processor causes the loss of critical spatial details. By borrowing the tiling methodologies used in high-precision agriculture where massive fields are analyzed piece by piece and combining them with distributed edge computing, this project proposes a scalable solution for real-time Human Presence Detection (HPD). We chose HPD as our primary focus because detecting people in large, open spaces requires the high-level granularity that traditional downsampling destroys. By splitting a high-resolution frame into smaller tiles and distributing the workload across a network of resource-constrained Microcontroller Units (MCUs), we maintain the precision needed to identify human subjects without the need for expensive, centralized hardware. This approach not only ensures accuracy in expansive environments but also keeps data processing local, addressing the latency and privacy concerns essential to modern monitoring systems.

1.1. Background

Standard CNN architectures like You Only Look Once (YOLO) or Faster Regions with Convolutional Neural Network (R-CNN) typically use an input resolution of 416×416 or 640×640. In high-resolution domains like surveillance or precision agriculture, an image can be 4000×3000 pixels.

The Problem of Vanishing Detail

In high-resolution surveillance tasks, the standard practice of resizing input images to fit fixed-size CNN architectures (e.g., 224×224 or 640×640) introduces a critical bottleneck known as Spatial Information Attrition. When an Ultra-high Definition (UHD) frame of 4000-pixel width is downsampled to 400 pixels, the transformation results in a 90% reduction in linear resolution and a 99% loss of total pixel data.

This geometric reduction disproportionately affects small-scale objects. For instance, a human subject occupying a 40×80 pixel area in the original frame is reduced to a mere 4×8 pixel fragment. Such a representation falls significantly below the minimum Effective Receptive Field (ERF) required for a CNN to extract discriminative features like limb orientation or torso contours [**?**]. At this scale, the high-frequency spatial components that define humanness are discarded, leaving the network to attempt classification based on aliased, low-frequency blobs that are indistinguishable from environmental noise.

Lessons from High-Precision Agriculture

Solution to the problem of vanishing detail exists in high precision agricultural models. The Small Object Detection (SOD) problem is a cornerstone of precision agriculture, where satellite and Unmanned Aerial Vehicle (UAV) imagery must be analyzed to identify individual crops, pests, or irrigation leaks. In these scenarios, the objects of interest often occupy less than 0.1% of the total image area [**?**].

To solve this, agricultural models employ a Tiling (or Patch-based) Inference strategy [**?**], [**?**]. Instead of global resizing, the image is partitioned into a grid of overlapping tiles. This preserves the Ground Sampling Distance (GSD); the physical distance on the ground represented by a single pixel ensuring that the spatial features required for detection remain intact. Research in this field, such as the Slicing Aided Hyper Inference (SAHI) framework [**?**], demonstrates that tiling can increase the average precision of small object detection by over 20-30% compared to standard resized inference.

But these solutions are often specialized and computationally complex, typically designed for offline processing on powerful workstations rather than real-time edge deployment.

Growing AI and Evolving Edge Computing

The rapid proliferation of Artificial Intelligence (AI) has led to a paradigm shift in how data is processed, moving from centralized cloud architectures to the Edge. As of 2025, it is estimated that over 75% of enterprise-generated data is created and processed outside traditional data centers [**?**]. This evolution is driven by the necessity for real-time responsiveness in applications such as autonomous surveillance and industrial monitoring, where the latency of round-trip cloud communication is no longer acceptable [**?**].

Modern edge computing involves a decentralized network of edge nodes ranging from Internet of Things (IoT) gateways to specialized hardware accelerators like NVIDIA Jetson modules that perform inference locally. However, the increasing complexity of state-of-the-art CNNs often exceeds the memory and thermal envelopes of a single edge device. This has birthed the concept of Distributed Edge Inference, where a singular heavy workload is partitioned across multiple heterogeneous nodes [**?**].

1.2. Motivation

The motivation for this research stems from the widening gap between the increasing resolution of modern image sensors and the static input constraints of high-performance deep learning models. While 4K and 8K cameras are becoming standard in security

and industrial monitoring, the hardware tax required to process these images at native resolution is prohibitive for most edge-deployed systems. Current solutions generally follow one of two suboptimal paths: aggressive downsampling, which sacrifices the high frequency spatial data necessary to detect small or distant subjects, or expensive hardware scaling, which relies on power hungry Graphics Processing Units (GPUs) that are unsuitable for remote or battery operated environments. There is a critical need for a middle ground—an architecture that preserves the raw detail of the sensor while distributing the computational burden across a modular network.

1.3. Problem Definition

The core challenge addressed in this research is the Information-Computation Paradox in high-resolution computer vision. As sensor technology advances, the gap between captured data density and real-time processing capability continues to widen. This problem is formally defined through three primary constraints:

The Scaling Bottleneck and Receptive Field Limitations: Most state-of-the-art CNN architectures, such as ResNet or YOLO variants, utilize fixed input dimensions (e.g., 416×416 or 640×640). When a 4K UHD stream is force-fitted into these dimensions, the resulting bilinear or bicubic interpolation acts as a low-pass filter, effectively erasing the high-frequency components that define small-scale objects. Consequently, the network's effective receptive field [**?**] becomes larger than the object itself, leading to a failure in feature activation.

Resource Heterogeneity and Memory Constraints: Edge devices often lack the unified memory architecture required to load large-scale models alongside high-resolution frame buffers. A monolithic inference approach on a single node leads to Out of Memory (OOM) errors or thermal throttling. There is currently a lack of standardized frameworks that allow a single inference task to be decomposed spatially across a cluster of heterogeneous low-power devices [**?**].

The Aggregation Challenge: Splitting an image into tiles introduces boundary artifacts. If a human subject is positioned across the intersection of four tiles, no single node possesses the complete spatial context to identify the subject. The problem, therefore, is not only how to distribute the tiles but how to design an Aggregator Node capable of fusing partial feature maps (logits or latent vectors) into a coherent global detection without re-introducing the original computational bottleneck [**?**].

1.4. Objectives

- To develop a light-weight model architecture to process a high resolution image by dividing into low-resolution tiles.
- To detect human presence by implementing the developed model using distributed architecture of resource constrained edge devices.

1.5. Scope and Applications

The scope of this project is centered on the intersection of distributed systems and embedded computer vision. While the methodology is developed using HPD as the primary use case, the underlying architecture for spatial partitioning and feature aggregation is designed to be model-agnostic.

By enabling high-resolution sensing on low-cost, low-power hardware, this project opens new avenues for several critical domains:

- **High-Precision Industrial Safety:** Monitoring large-scale factory floors where small-scale anomalies (e.g., a hand near a restricted gear) must be detected at high resolution without the cost of high-end industrial Personal Computerss (PCss).
- **Privacy-Preserving Smart Homes:** Since tiles are processed locally and only feature maps (not raw images) are aggregated, the system inherently provides a layer of data privacy, making it ideal for residential presence sensing and elderly care monitoring.
- **Energy Management in Heating, Ventilation, and Air Conditioning (HVAC) Systems:** In large-scale commercial buildings or smart offices, precise occupancy detection is critical for optimizing HVAC loads. Traditional Passive Infrared (PIR) sensors often fail to detect stationary occupants, leading to energy inefficiencies. A distributed, tile-based CNN can monitor vast open-plan spaces with high fidelity, detecting occupants even in seated or obscured positions. By feeding real-time presence data into a Building Management System (BMS), the HVAC system can dynamically adjust airflow and temperature in specific zones, significantly reducing the building's carbon footprint and operational costs.

The boundaries of this study are limited to static or slow-moving subjects where the latency of distributed aggregation does not exceed the temporal changes in the scene.

2. LITERATURE REVIEW

The trajectory of smart building sensor technology is increasingly moving toward complex computer vision solutions to overcome the inherent failures of traditional sensing systems. The evolution from basic detection technologies like Passive Infrared (PIR) and Bluetooth Low Energy (BLE) is driven by the demand for superior accuracy and the critical need to differentiate between simple movement and actual physiological presence. Traditional PIR sensors are highly sensitive to thermal movement but fundamentally lack the ability to confirm the presence of a stationary individual, resulting in unreliable occupancy data for HVAC or smart lighting. While hybrid systems combining PIR and Radar have emerged to reduce false alerts, they still struggle with high-fidelity requirements [**?**]. High-fidelity presence detection requires the capacity to sense subtle physiological indicators such as posture shifts or minute breathing patterns. Literature benchmarks indicate that Passive Infrared (PIR) sensors have a stationary detection reliability of approximately 0.0%, making them ineffective for modern energy management needs when compared to PC-based models that achieve an 85.0% Mean Average Precision (mAP). This project's computer vision approach is designed to replace these specialized sensors by analyzing visual light patterns for high-frequency components that correspond to human activity, resolving the stationary detection issue by maintaining a presence signal even when movement is minimal.

To bridge the gap between complex deep learning models and the extreme hardware limitations of edge microcontrollers, quantization is employed as a critical optimization strategy. By reducing the precision of the model's numerical values, it becomes possible to fit sophisticated human detection algorithms into a very small memory footprint [**?**, **?**]. The industry standard involves converting 32-bit floating-point (FP32) weights into 8-bit integers (INT8), providing a four-fold reduction in memory. For models like MobileNetV2, Post-Training Quantization (PTQ) has been shown to retain up to 99.7% of baseline accuracy. While extreme 4-bit (INT4) quantization allows for even smaller models, it often introduces a loss in precision that requires recovery techniques like layer-wise equalization. Modern approaches, such as those used in Q-CNN, prioritize response fidelity rather than just weight error, achieving up to 20x model compression with less than a 1% drop in performance [**?**]. For this project, quantization is a necessity to meet the strict 512 KB SRAM ceiling of the ESP32-S3, as standard single-node edge solutions typically struggle with throughputs of only 1.0–1.5 FPS and inference latencies ranging from 800 ms to 1200 ms.

To deploy these models effectively, the literature emphasizes optimizing both the arrangement of layers and the nature of the weights themselves. The SqueezeNet architecture achieves high accuracy with minimal parameters through the "Fire Module,"

which uses 1×1 "Squeeze" filters to reduce channels before they reach expensive 3×3 "Expand" filters [**?**]. Complementing this, Local Binary Convolutional Neural Networks (LBCNN) represent a paradigm shift toward fixed stochastic projections. Rooted in Local Binary Patterns (LBP), LBCNN replaces standard learnable filters with fixed, sparse, binary kernels. In an LBC layer, the response is calculated as a learnable linear combination of these pre-defined binary filters, providing an immediate 9x reduction in parameters for 3×3 kernels, scaling up to 169x in deeper architectures. Because binary kernels are compatible with bitwise operations, LBCNN facilitates lower latency and reduced power consumption, allowing deep models to operate within the 512 KB SRAM limits of devices like the ESP32-S3 [**?**].

To avoid Out of Memory (OOM) crashes and overheating, which occur when a single chip handles massive 4K images, the literature suggests a distributed approach. While "Model Partitioning" exists, "Data Partitioning" or Tiling—which this project uses—is preferred for high-resolution data [**?**]. This horizontal split involves cutting the image into a grid of independent tiles, allowing each node to run the full model in parallel and preserving the fine details usually lost when an image is shrunk. However, distributed systems must mitigate "Communication Overhead"; if the transfer time (T_{comm}) exceeds the computation time (T_{comp}), the system slows down [**?**]. To solve this, the project utilizes the ESP-NOW protocol, offering low-latency peer-to-peer communication (1.115 ms) and handling limited bandwidth (500–771 KB/s) by transmitting small result packets rather than raw images. This hierarchy includes an Input Node (ESP32-CAM) for tile partitioning, Edge Computing Nodes (ESP32-S3) for parallel inference, and an Aggregator Node to reconcile detections.

Current research highlights two major gaps that this project aims to bridge. First, the Information Loss Gap occurs because most systems resize 4K streams to fit tiny models, acting as a blur filter that erases subtle human features. Second, the High-End Hardware Gap exists because while methods like Slicing Aided Hyper Inference (SAHI) prove that tiling improves small object detection by 20–30% [**?**], these are typically designed for expensive workstations. By implementing a Distributed Tiling architecture and using LBCNNs to shrink the model size, this project achieves high-definition accuracy previously reserved for expensive PCs. This approach ensures total data privacy by keeping processing local and provides a low-cost, scalable solution for industrial safety and smart energy management.

3. REQUIREMENTS ANALYSIS

The development of a distributed, tile-based HPD system requires a multi-faceted analysis of hardware constraints, software dependencies, and network reliability. This section outlines the essential requirements to ensure the feasibility of the project.

3.1. Hardware Requirements

Since the project targets the ESP32-S3 platform, the hardware selection is dictated by the need for AI acceleration and power efficiency.

- **Camera Module:** A high-resolution camera module(OV2640) capable of capturing images at higher resolution than traditional image processing systems, which will then be partitioned.
- **Input Node:** A master node responsible for taking images from the image sensor; partitioning them into multiple tiles and distributing them to edge nodes.
- **Edge Computing Nodes:** Multiple ESP32-S3 MCUs are required. These must support the $XtensaLX7$ vector instructions to handle the tile-level CNN inference.
- **Aggregator Node:** A simple ESP32 responsible for collecting feature vectors and executing the final classification layer.
- **Communication Interface:** A low-latency communication system for inter-node communication Espressif Non-Beaconing Wirelesss (ESP-NOWs).

3.2. Software & Framework Requirements

To achieve ultra-low bit quantization and distributed processing, the following software stack is required:

- **Development Environment:** Arduino IDE using C/C++.
- **AI Inference Library:** Espressif Deep Learning library (ESP-DL) to utilize high-performance kernels for quantized 8-bit or 16-bit operations.
- **Model Optimization:** TensorFlow Lite for Microcontrollers (TFML) for initial model training and conversion, followed by Quantization-Aware Training (QAT).
- **Tiling Logic:** A custom C++ pre-processing module to handle spatial partitioning and memory-safe buffer management for individual tiles.

3.3. Non-Functional Requirements

- **Latency:** The total time must be low enough for real-time processing.
- **Scalability:** The system should support a variable number of nodes depending on the resolution and required Frames Per Second (FPS).

- **Robustness:** In the event of a node failure, the system should gracefully degrade, relying on the remaining tiles to provide partial occupancy data.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. Hardware Used

The core hardware of this project is built around ESP32-family MCUs. It is chosen because they are low-cost, low-power, and capable of running edge-level machine learning tasks. This system follows a distributed architecture. Here, image capturing, processing and decision-making are separated across different ESP32 devices to overcome memory and processing limitations of a single microcontroller. This hardware design allows the system to perform real-time HPD without relying on cloud servers, making it faster, more private, and more energy efficient.

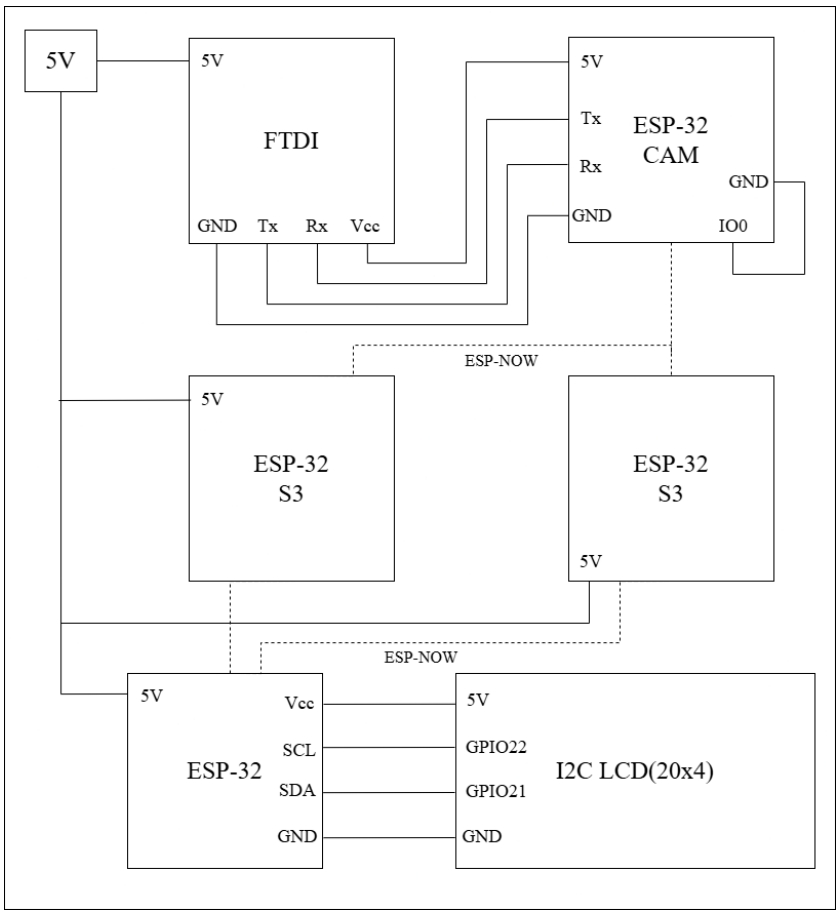

Figure 4-1. Wiring Interface between Programming (FTDI), Processing (ESP32-S3), and Visualization (I2C LCD) Units.

The hardware architecture as shown in 4-1 consists of a distributed processing network designed for real-time data acquisition and visualization. The system is partitioned into three primary functional blocks: the Sensing Node (ESP32-CAM), the Processing Cluster (ESP32-S3 units), and the Output Interface (I2C LCD connected via ESP32)

4.1.1. ESP32-CAM

The Espressif Systems (ESP)-CAM module is used as the input hardware of the system. Its main role is to capture visual data from the environment. Here, the camera continuously captures images of 224 × 224 at a controlled rate of (.....). Since color images require more memory and computation, the ESP-CAM converts the captured rgb images into grayscale format. After this, the high-resolution image is divided into smaller tiles of 96 × 96 with overlap of 25% between adjacent tiles. These smaller image tiles are easier to process and then transmit to other ESP devices using ESP-NOW protocol for further processing. Here, the tiles are evenly distributed between the two esp32s3 devices when the total number of tiles is even; if the number of tiles is odd, the first node is assigned one additional tile.

4.1.2. USB to TTL converter (FTDI)

The USB to TTL (FTDI) converter is used to connect the ESP32-CAM boards to a laptop for programming. It is because ESP32-CAM does not have a built-in USB interface for programming or serial communication. The FTDI module acts as a bridge between them since a laptop communicates using USB and the ESP32 uses TTL serial (UART) signals. In this project, the FTDI also supplies 5V power from the laptop via USB, which removes the need for an external power source.

Connect GPIO0 to GND in ESP32-CAM for flashing. This indicates that the ESP32-CAM is ready to receive new code. After uploading the code, we can disconnect GPIO0 from GND.

Table 4-1. Wiring the FTDI to ESP32-CAM

FTDI	ESP32-CAM
VCC	5V
GND	GND
RX	U0T
TX	U0R

4.1.3. ESP32-S3

The ESP32-S3 MCUs act as the processing or inference nodes in the system. Each ESP32-S3 receives a small image tile from the ESP32-CAM and runs a Tiny Machine Learning (TinyML)-based HPD model on it. These boards are specially selected because they are optimized for AI tasks and support low-bit operations. These are essential for running neural networks within limited memory. Here, each ESP32-S3 processes its assigned tile independently and determines whether human features are present. Instead

of sending large image data back, it sends only a compact result, such as a confidence score and tile location, which minimizes communication overhead.

4.1.4. ESP32

The ESP32 board is used as the aggregation node of the system. It is used to collect the results from all ESP32-S3 and uses attention weight from all tiles to compute attention score using SoftMax function and then uses the attention score to perform weighted sum of feature maps of tiles. Then, it passes the new feature map through classifier layer to compute the confidence score. This confidence score is to form a final decision about human presence in the entire scene.

4.1.5. I2C LCD Display

In this system, the I2C LCD display is used to show the final output of the project. After the ESP32-S3 boards finish processing the image tiles and send their results, the ESP32 combines all the information and decides whether a human is present or not. This final decision is then shown on the LCD screen as **"Human Detected"** or **"No Human Detected".** The I2C LCD has been chosen because it needs only two wires for communication. It keeps the wiring simple and saves GPIO pins on the ESP32.

Table 4-2. Wiring the I2C LCD to ESP32

I2C LCD Pin	ESP32 Pin	Purpose
VCC	3.3V or 5V	Power Supply
GND	GND	Ground Connection
SDA	GPIO21	Data Line
SCL	GPIO22	Clock Line

4.1.6. Power Supply

All the hardware components in the system are powered using a laptop via USB connections. The laptop provides a stable 5V power supply to the ESP32-CAM, ESP32-S3, and ESP32 boards during development and testing. This simplifies the setup, eliminates the need for external power adapters, and allows easy monitoring and debugging through serial communication.

4.2. Hardware Specifications and system topology

The project utilizes a decentralized, multi-node architecture deployed on ESP32 and ESP32-S3 MCUs. To optimize throughput and manage resource constraints, the system is organized into a three-tier hierarchical structure.

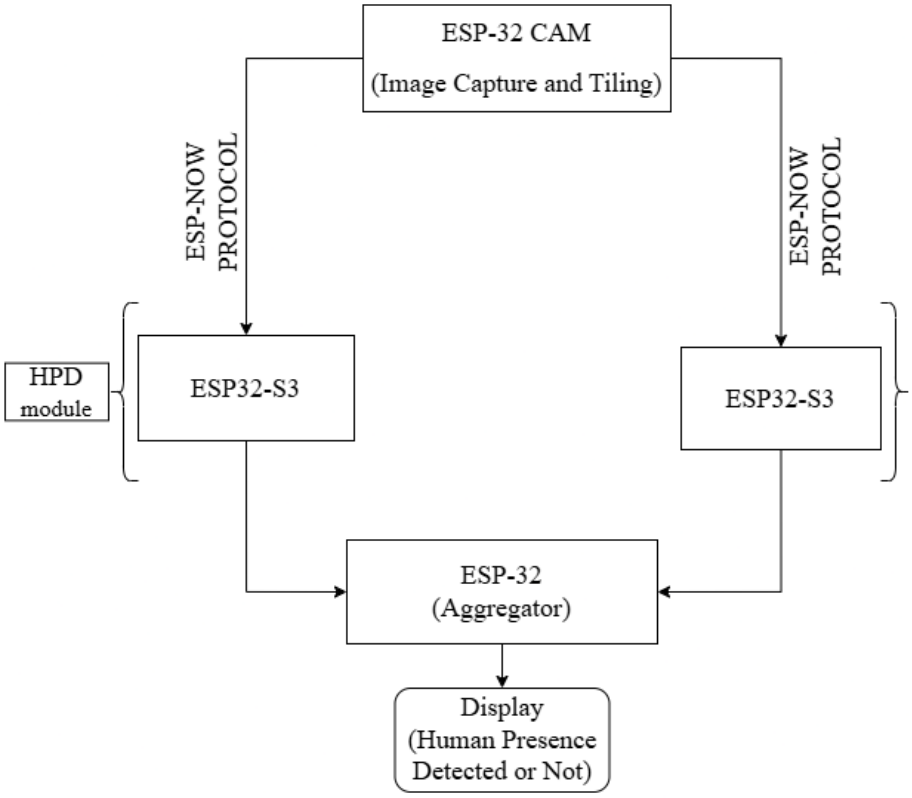

Figure 4-2. System Architecture showing main components and their interactions.

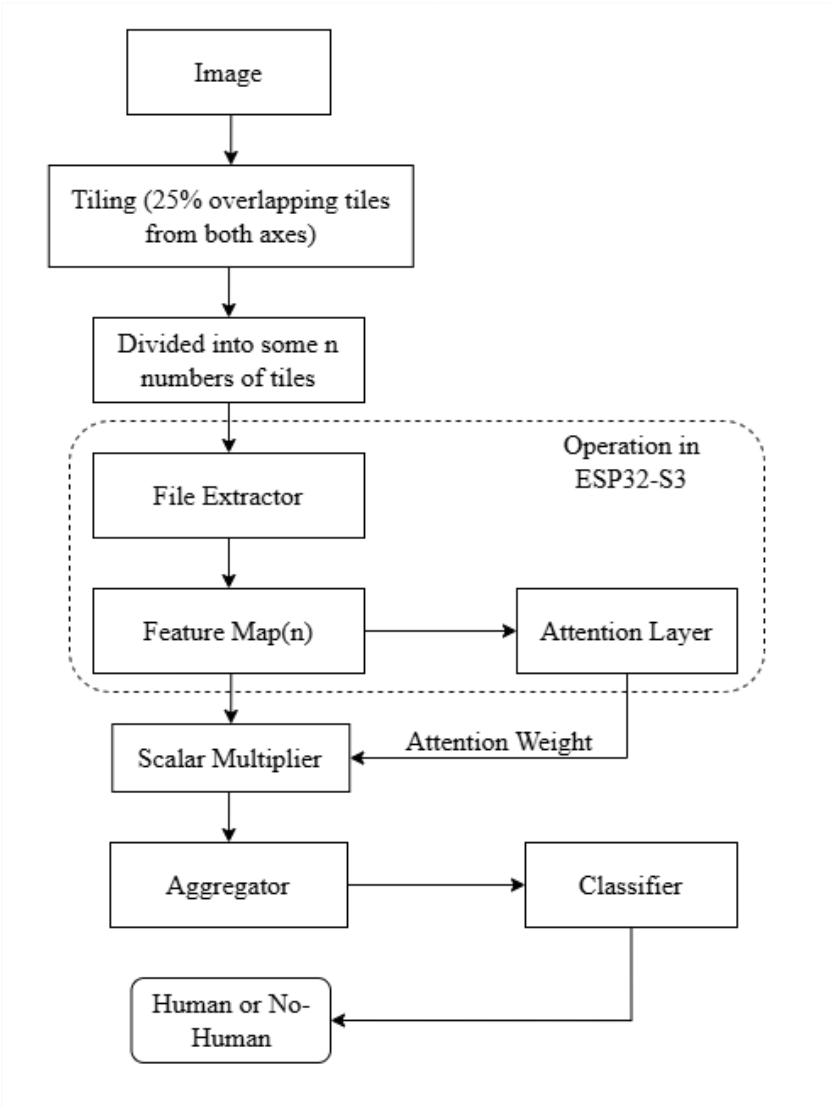

Figure 4-3. Detailed Data Flow Diagram of the Tiling and Inference Pipeline.

4.2.1. Input Layer

Centered around an ESP32-CAM [**?**], this layer manages high-bandwidth data acquisition and initial pre-processing. Its primary role is to prepare and distribute workloads to the processing tier through the following pipeline:

Luminance Conversion (Grayscale)

Processing three-channel RGB data significantly inflates the memory buffer requirements and computational complexity ($O(3n)$ vs $O(n)$). The captured frame has to be converted to an 8-bit grayscale format using a weighted luminance formula as given in (4-1) [**?**].

$$P(y = 1 \mid x) = \frac{e^{z_{\text{person}}}}{e^{z_{\text{bg}}} + e^{z_{\text{person}}}} \tag{4-1}$$

Where z_{person} and z_{bg} are the logits for the "person" and "background" classes, respectively. This conversion reduces the data size by two-thirds, significantly easing the memory and computational load on the subsequent processing nodes. This reduces the input data size by 66.6%, freeing up critical heap memory (Static Random-Access Memory (SRAM)) for the tiling and distribution buffers.

$$Y = 0.299R + 0.587G + 0.114B \tag{4-2}$$

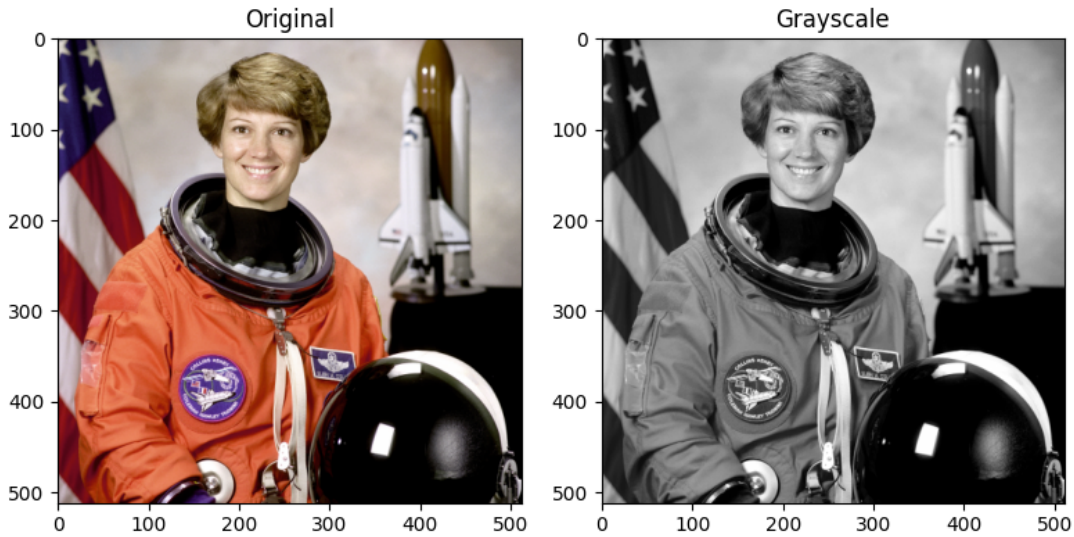

Figure 4-4. RGB to Grayscale
[?]

Spatial Partitioning Distribution

The process begins with the Tiling module, which extracts n overlapping tiles from the source image. This step is critical for maintaining high-resolution features on resource-constrained hardware. By using a 25% overlap across both axes, the system ensures that human subjects positioned at tile boundaries are not bisected, preserving spatial continuity for the feature extractor.

4.2.2. Communication Protocol Implementation

The ESP protocol will be the communication backbone, selected for its highly efficient, peer-to-peer data exchange capabilities and low-latency feedback. While ESP is efficient, its throughput is limited (observed maximums up to 771 Kilobyte (KB)/s). Therefore, the implementation will rely on efficient data packing, ensuring that tile data transfer maximizes the available throughput and that result packets adhere to the small packet size limits (< 250 bytes for control and aggregation data). The ability to transfer data

quickly is paramount, as the latency gained by parallel processing can be quickly lost if the communication time dominates the total system latency.

4.2.3. HPD Model Layer (Distributed Inference)

- Parallel Processing: Each node receives a specific low-resolution tile from the Input Layer.
- Local Inference: Nodes perform localized HPD analysis by modeling human presence as a Bernoulli distribution ($y \in \{0, 1\}$). The CNN utilizes a Softmax function to map raw model outputs (logits) to a posterior probability as given in (4-1) :
- Node ID: A 1-byte identifier used to map the tile's location within the larger 224×224 frames.
- Quantized Score (Q): The posterior probability is scaled to a single-byte integer ($Q = P \times 255$). This significantly reduces communication overhead compared to sending bulky decimal numbers [**?**].
- Local Bounding Box (B_{local}): Spatial coordinates $[x, y, w, h]$ that pinpoint exactly where "person-like" features were detected within that specific tile [**?**].

4.2.4. Output Layer (Aggregation & Logic)

As the final stage of the architecture, this layer serves as the central decision-making layer:

- **Data Synthesis:** It aggregates the distributed output vectors from the HPD Model Layer.
- **Final Prediction:** By synthesizing the localized data, the Output Layer produces the final global HPD prediction for the entire captured frame.

4.3. Model Selection and Optimization Pipeline

4.3.1. Base Model Selection and Training

1. Local Binary Convolution based Architecture

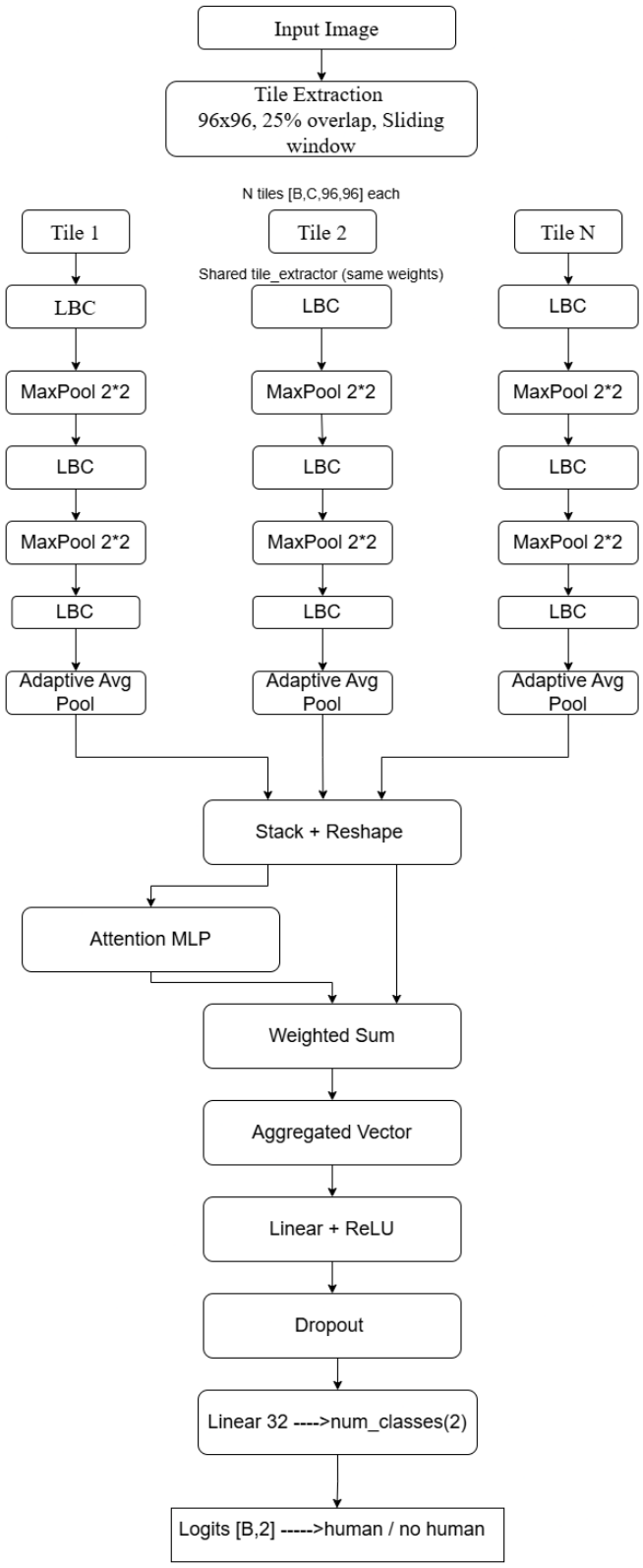

Figure 4-5. Proposed Multi-Tile Attention-Based CNN Architecture

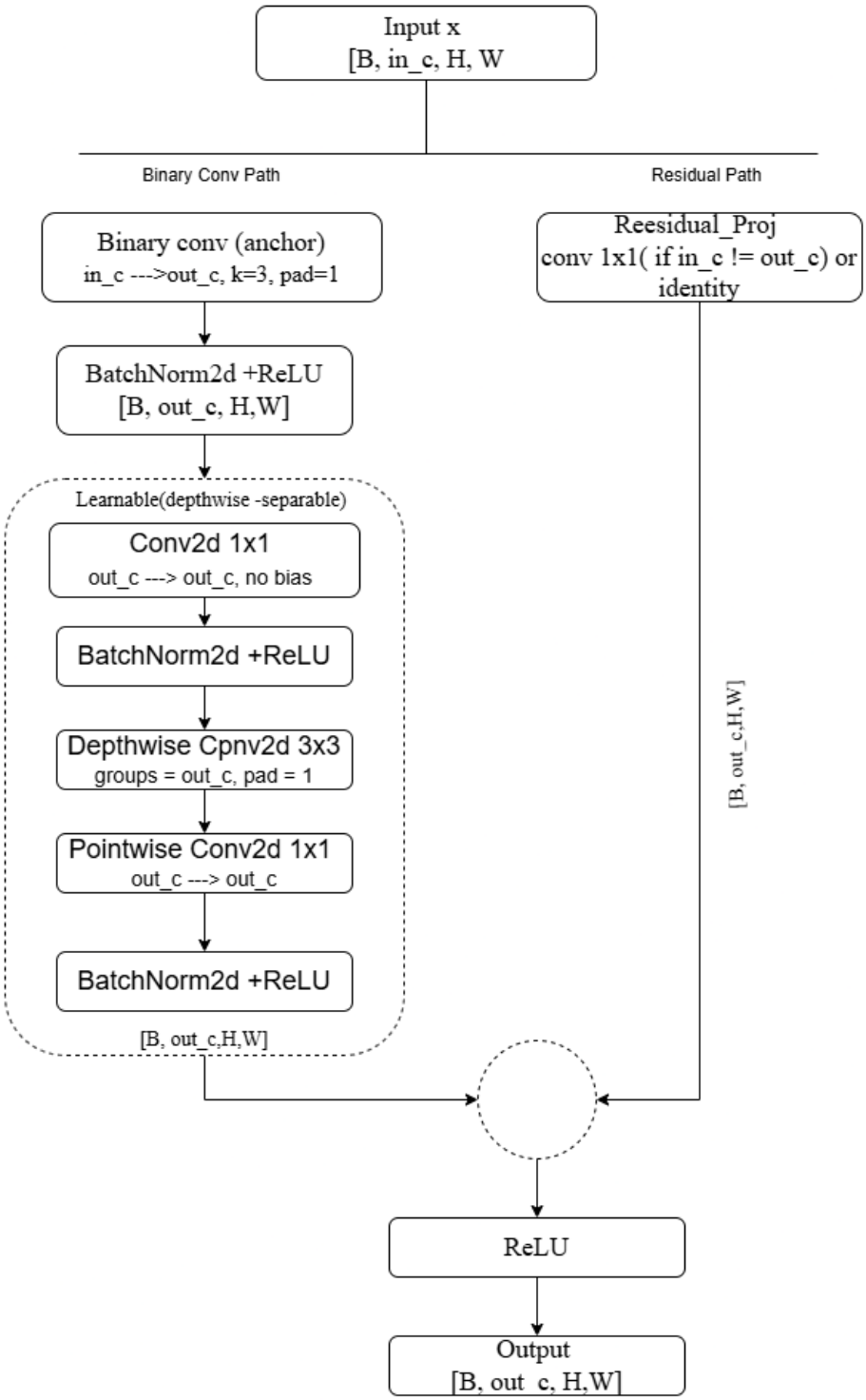

Figure 4-6. Detailed Schematic of the Local Binary Convolution Module

1.1 LBCLayer Overview

The LBCLayer is a hybrid convolutional layer designed to combine fixed binary convolutions (anchors) for efficient sparse structural feature capture with a learnable 1×1

convolution for task-specific adaptation. It is particularly suitable for resource-efficient networks and sparse feature representations, inspired by research on Local Binary Convolutions (LBC) and sparse binary kernels.

Mathematically, for an input $x \in \mathbb{R}^{N \times C_{in} \times H \times W}$, the output is defined as:

$$x_{\text{out}} = \text{Conv1x1}(\text{ReLU}(\text{Conv}_{\text{binary}}(x))) \tag{4-3}$$

Where:

- $\text{Conv}_{\text{binary}}$ uses fixed ternary weights $\in \{-1, 0, 1\}$.
- Conv1x1 is trainable and combines channels efficiently.

1.2. LBCLayer Components

1.2.1. Fixed Anchor Convolution

The purpose of this component is to extract general features without learning, thereby reducing the parameter count.

- **Weight generation**: Weights are sampled from a normal distribution $W \sim \mathcal{N}(0, 1)$ and then thresholded to obtain sparsity s:

$$\text{binary_weight}_{i,j,k,l} = \begin{cases} 1 & \text{if } W_{i,j,k,l} > \text{threshold} \\ -1 & \text{if } W_{i,j,k,l} < -\text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

- The sparsity parameter s controls the fraction of zero weights.
- These weights are stored as a buffer (`register_buffer`), meaning they are not updated during training, ensuring stability and a low memory footprint.

The key idea is to capture general directional/edge patterns efficiently, similar to handcrafted filters like Sobel, but stochastically generated.

1.2.2 Non-linearity (ReLU)

After the fixed binary convolution, ReLU is applied: $x' = \text{ReLU}(x)$. This introduces non-linearity to approximate complex functions while preserving sparse activations and improving computational efficiency since many values remain zero.

1.2.3 Learnable 1x1 Convolution

The 1×1 kernel combines sparse feature maps into task-relevant channels while keeping computational costs low.

$$x_{\text{final}} = W_{1\times1} * x' \tag{4-4}$$

Only this part is trainable, allowing the layer to adapt to specific tasks like human detection.

1.3 Forward Pass Flow

For a single LBCLayer, the flow is as follows:

1. **Input tensor**: $x \in \mathbb{R}^{N\times C_{in}\times H\times W}$.
2. **Binary convolution**: $x_1 = \text{Conv}_{\text{binary}}(x)$.
3. **ReLU activation**: $x_2 = \text{ReLU}(x_1)$.
4. **Learnable 1x1 convolution**: $x_{\text{out}} = \text{Conv1x1}(x_2)$.
5. **Output**: $x_{\text{out}} \in \mathbb{R}^{N\times C_{\text{out}}\times H\times W}$.

1.4 Advantages of LBCLayer

- **Efficiency**: Fixed sparse weights reduce multiply-adds, and 1×1 learnable convolutions are computationally cheap.
- **Stability**: Binary anchors are not updated, which avoids noisy gradients; the network only optimizes the final 1×1 convolution.
- **Sparsity Control**: The sparsity parameter allows control over network complexity and feature density, encouraging sparse representations that improve generalization.
- **Compatibility with Tiling**: LBCLayer is ideal for tile-based architectures like the HumanDetector, where each tile can be processed independently and then stitched.
- **Biologically Inspired**: Sparse, binary-like filters resemble early visual cortex processing and are useful for detecting edges, textures, and local patterns.

1.5 Mathematical Summary

For an input x, the summary of the operation and gradients is:

$$\begin{aligned}
\text{Anchor weight: } A &= f_{\text{binary}}(W, s) \\
x_1 &= x * A \\
x_2 &= \text{ReLU}(x_1) \\
x_{\text{out}} &= x_2 * W_{1\times1}
\end{aligned}$$

Gradients: $\frac{\partial L}{\partial W_{1\times1}} \neq 0$, whereas $\frac{\partial L}{\partial A} = 0$. Here, $*$ denotes convolution, s is the sparsity threshold, and A represents the fixed binary anchor weights.

1.6 Overview of HPD

The HPD (HumanDetector) is a **tile-based convolutional network** designed to detect humans in images. Its key characteristics:

1. **Input:** Grayscale images $x \in \mathbb{R}^{N\times1\times H\times W}$
2. **Tiling:** Split images into tile_rows × tile_cols non-overlapping patches.
3. **Feature Extraction per Tile:** Uses LBCLayer to extract sparse yet task-adapted features.
4. **Aggregation / Stitching:** Concatenates all tile features into a single vector.
5. **Classifier Head:** Dense layers to predict **num_classes** logits (e.g., Human vs Not Human).

Mathematically:

$$f_\theta(x) = \text{Classifier}\left(\text{Concat}_{i,j}(\text{TileExtractor}(x_{i,j}))\right) \tag{4-5}$$

Where $x_{i,j}$ is the (i, j)-th tile.

1.7 Tiling Stage

The tiling stage prepares the input for localized processing.

- **Input:** $x \in \mathbb{R}^{N\times1\times H\times W}$
- **Parameters:** tile_rows, tile_cols
- **Tile size:**

$$h = \frac{H}{\text{tile_rows}}, \quad w = \frac{W}{\text{tile_cols}} \tag{4-6}$$

- **Split input into tiles:**

$$x_{i,j} = x[:, :, i \cdot h : (i+1) \cdot h, j \cdot w : (j+1) \cdot w] \tag{4-7}$$

Purpose:

- Localizes feature extraction → each tile acts as a **node** (MCU-like processing).
- Improves **spatial granularity** for detecting humans.
- Tiles are **independent**, can be processed in parallel.

1.8 Tile Feature Extractor (LBC-based)

Each tile passes through the `tile_extractor` which follows this architecture:

- **LBC Layer 1:** $1 \to 16$ channels, sparsity = 0.9.
- **MaxPool2d:** Kernel size 2, halves spatial dimensions.
- **LBC Layer 2:** $16 \to 32$ channels, sparsity = 0.9.
- **AdaptiveAvgPool2d:** Reduces spatial dimension to 1×1.

For a single tile $x_{i,j}$:

$$x' = \text{LBCLayer}(x_{i,j}) = \text{Conv1x1}(\text{ReLU}(\text{Conv}_{\text{binary}}(x_{i,j}))) \tag{4-8}$$

1.9 Aggregation / Stitching

After feature extraction, the tile outputs are combined:

1. **Flatten each tile output:** $f_{i,j} \in \mathbb{R}^{32}$
2. **Concatenate all tiles:** If num_tiles = tile_rows × tile_cols:

$$f_{\text{combined}} = [f_{1,1}, f_{1,2}, \ldots, f_{\text{tile_rows,tile_cols}}] \in \mathbb{R}^{32 \cdot \text{num_tiles}} \tag{4-9}$$

This acts like **graph aggregation**, treating each tile as a node.

1.10 Classifier Head

The classifier takes the concatenated vector and applies a MLP:

- **Input:** $f_{\text{combined}} \in \mathbb{R}^{32 \cdot \text{num_tiles}}$
- **Layers:** Linear($32 \cdot$ num_tiles $\to 64$) $\to$ ReLU $\to$ Linear($64 \to$ num_classes)
- **Output:** Logits for classification:

$$\hat{y} \in \mathbb{R}^{N \times \text{num_classes}} \tag{4-10}$$

Uses **CrossEntropyLoss** during training:

$$L = -\sum_{c=1}^{\text{num_classes}} y_c \log \text{softmax}(\hat{y}_c) \tag{4-11}$$

1.11 Computational Efficiency

The LBCLayer reduces Floating Point Operationss (FLOPss) significantly:

$$\text{FLOPs} \approx C_{in} \cdot C_{out} \cdot k^2 \cdot H \cdot W \tag{4-12}$$

- **Binary conv:** Multiplications become simple additions/subtractions.
- **1x1 conv:** Provides lightweight mixing of channels.
- **Tile-based processing:** Can be parallelized across multiple nodes.

1.12 Mathematical Summary of Forward Pass

1. **Tile splitting:**

$$\{x_{i,j}\}_{i,j=1}^{R,C} = \text{Tile}(x) \tag{4-13}$$

2. **Feature extraction (per tile):**

$$f_{i,j} = \text{AdaptiveAvgPool2d}(\text{LBCLayer2}(\text{MaxPool2d}(\text{LBCLayer1}(x_{i,j})))) \tag{4-14}$$

3. **Flatten + concatenate:**

$$f_{\text{combined}} = \text{Concat}([f_{i,j}]) \tag{4-15}$$

4. **Classification:**

$$\hat{y} = \text{Linear}_2(\text{ReLU}(\text{Linear}_1(f_{\text{combined}}))) \tag{4-16}$$

5. **Loss:**

$$L = -\sum_{c=1}^{\text{num_classes}} y_c \log \text{softmax}(\hat{y}_c) \tag{4-17}$$

1.13 Advantages of HPD Architecture

1. **Sparse and efficient:** LBC reduces compute and memory footprint.
2. **Tile-based learning:** Learns local patterns for small human parts.
3. **Flexible:** Can adapt tile size and sparsity based on resources.
4. **Parallelizable:** Each tile is processed independently.
5. **Low-data regime:** Predefined anchors reduce training requirements.

2.Modified MobileNetV2 Architecture

The proposed system utilizes a CNN based on a modified mobilenetv2 backbone, specifically optimized for binary classification tasks where $f_\theta : \mathbb{R}^{N\times1\times H\times W} \to \mathbb{R}^{N\times2}$. The architecture is designed to balance high-level feature extraction with the strict hardware constraints of MCUs.

2.1.Grayscale Adaptation and Transfer Learning

To minimize memory consumption and computational latency, the model is adapted to process single-channel grayscale images. Since the pretrained weights were originally optimized for rgb data, the first convolution layer's weights ($W \in \mathbb{R}^{C_{out} \times 3 \times k \times k}$) are averaged across the channel dimension:

$$W_{\text{gray}} = \frac{1}{3}(W_R + W_G + W_B) \tag{4-18}$$

This strategy preserves low-level features, such as edge and texture detectors, while allowing the model to function on simplified SRAM-efficient inputs.

2.2.Inverted Residuals and Efficiency

The feature backbone leverages Inverted Residual Blocks, which utilize depthwise separable convolutions to reduce parameters. This design follows a "Narrow → Wide → Narrow" sequence:

$$x \rightarrow \text{Conv}_{1\times1} \rightarrow \text{DWConv}_{3\times3} \rightarrow \text{Conv}_{1\times1}$$

The computational cost reduction factor, compared to standard convolutions, is defined as $\frac{1}{N} + \frac{1}{D_k^2}$, making it ideal for edge deployment.

2.3.Freezing Strategy and GAP

To stabilize training on the 737-sample dataset, a freezing strategy is employed where the backbone parameters (θ_f) are kept static. Optimization only updates the task-specific head (θ_t):

$$\theta_t \leftarrow \theta_t - \eta \nabla_{\theta_t} L \tag{4-19}$$

Spatial features are then compressed using Global Average Pooling (GAP), which transforms the tensor from $\mathbb{R}^{N \times 1280 \times h \times w}$ to $\mathbb{R}^{N \times 1280}$ by calculating the spatial average:

$$z_c = \frac{1}{hw} \sum_{i=1}^{h} \sum_{j=1}^{w} x_{c,i,j} \tag{4-20}$$

2.4.Classifier Head and Loss Function

The final classification is performed by a narrow bottleneck head (1280 → 64 → 2) using ReLU activation and dropout (0.2 and 0.1) to prevent overfitting. The resulting logits $\hat{y}$ are evaluated using Cross-Entropy Loss:

$$L = -\sum_{i=1}^{2} y_i \log\left(\frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}\right) \tag{4-21}$$

The complete forward pass is sum marized as:

$$f_\theta(x) = W_2\text{ReLU}(W_1(\text{GAP}(\text{Backbone}(\text{Conv}_{gray}(x))))) \tag{4-22}$$

4.3.2. Pruning Strategy

A structured pruning strategy targeting channel reduction will be employed. Pruning decisions will be guided by importance scores derived from L1 magnitude or activation sensitivity. The objective is to significantly reduce the model's Multiply-Accumulate Operationss (macs) and memory footprint (measured in Megabyte (MB)) while retaining functional accuracy and to achieve a pruning ratio between 50% and 70% of the initial channels, minimizing the total number of arithmetic operations executed per tile inference.

4.3.3. Low-Bit Quantization Implementation

The baseline operational model will utilize Post-Training Quantization (PTQ) to convert the HPD model to symmetric 8-bit Integer (INT8) weights and asymmetric INT8 activations. This approach is selected for its demonstrated success in significantly reducing computational complexity while maintaining high classification reliability in resource-constrained deployments. Further optimization will involve the experimental deployment of 4-bit Integer (INT4) quantization for specific non-critical layers. This mixed-precision approach will push memory reduction to its architectural limits, targeting a final Flash footprint of < 1.0 MB.

To mitigate the precision loss associated with such aggressive compression, calibration datasets will be utilized in conjunction with layer-wise equalization techniques. These methods balance weight ranges across the custom architecture, aiming for maximum accuracy retention (targeting lesser than 90% of the pruned 32-bit Floating Point (FP32) baseline). This optimization strategy is essential to ensure that the model remains robust for presence detection while fitting within the 512 KB SRAM limit and achieving the system's 1.0 FPS throughput target.

The target inference time for the optimized custom CNN is set at < 70 Milliseconds (ms) per tile. This strict latency requirement is justified by the observation that even a standard INT8 MobileNetV2 on similar Xtensa cores requires 118 ms. The expectation is that the custom-designed, shallow architecture combined with structural pruning, INT4 quantization, and the ESP32-S3's specialized AI Vector instructions will significantly reduce the required mac operations. This makes it possible to execute local tile inference faster than the communication and synchronization window, effectively offsetting the transmission overhead of the ESP protocol and ensuring the distributed system maintains its target throughput.

4.4. Evaluation Metrics and Performance Framework

We use a multi-layered evaluation methodology to thoroughly evaluate the Distributed Multi-Agent Presence Detection System's efficiency. This framework measures the effectiveness of the Master-Worker architecture and its performance under stringent TinyML limitations, going beyond mere correctness.

4.4.1. Detection Reliability Matrix (The Deep Learning Layer)

The main goal is to fix the stationary detection issue that PIR sensors have. The following standard computer vision criteria will be used to assess our CNN model:

Recall (Sensitivity)

Measured especially for participants who are motionless to demonstrate the system's capacity to sustain presence signals in situations where movement is minimal. The Equation is as shown in equation (3-3):

$$Recall = TruePositive(TP)/(TP + FalseNegative(FN)) \tag{4-23}$$

Precision

To ensure environmental noise (shadows, HVAC airflow) does not trigger false positives in the model's inference. The Equation is as shown in equation (3-4):

$$Precision = TP/(TP + FalsePositive(FP)) \tag{4-24}$$

Mean Average Precision (mAP)@0.5 (Mean Average Precision)

The Common Objects in Context (COCO) dataset standards were used to evaluate mAP@0.5 which concentrated on the person class at a 50% Intersection over Union (IoU) criterion. This is the official Deep Learning object detection benchmark.

4.4.2. Distributed System Efficiency Matrix (The Computing Layer)

We must measure the benefits of distributing the inference burden among several ESP32-S3 nodes using the Distributed Adaptive Tiling (DAT) framework as this project makes use of a Distributed Cluster.

End-to-End Latency (total): The duration between the Master node's picture acquisition and the ultimate detection outcome.

System Throughput: FPSs is the unit of measurement for system throughput. 1.0 FPS is set as the standard for minimum real-time occupancy sensing due to the limitations of the ESP32-S3 and the overhead of tiling.

Communication Overhead: The proportion of the entire cycle used for local Deep learning computation as opposed to ESP tile transmission.

5. RESULTS AND ANALYSIS

The following report details the training and evaluation of a LBC architecture for HPD. The model was trained over 45 epochs using a `OneCycleLR` scheduler before early stopping was triggered.

5.1. LBCNN Architecture Performance

The utilization of LBC architecture for the task of human presence detection demonstrated the feasibility of a lightweight model with reliable accuracy on resource-constrained MCUs.

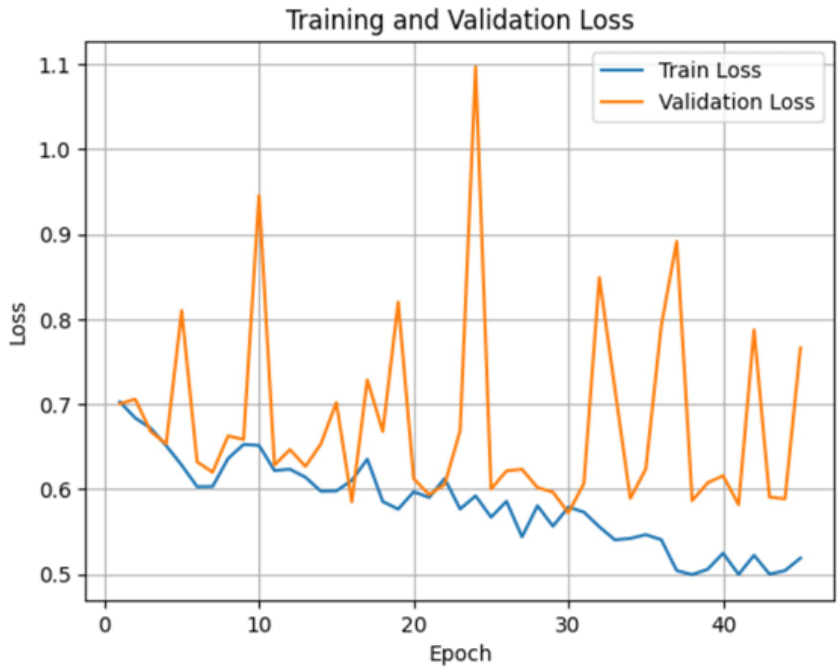

Figure 5-1. Training and validation loss over 45 epochs.

The model was trained for 45 epochs (with an early stopping patience of 15) on 737 training samples. As shown in Fig. 5-1, the training loss decreases consistently while the validation loss remains volatile, indicating moderate overfitting due to the limited dataset size. The best validation loss of **0.5721** was recorded at epoch 30.

Table 5-1. Training Run Summary

Metric	**Value**	**Epoch**
Best validation loss	0.5721	30
Best validation accuracy	77.17%	43
Final training accuracy	78.56%	45
Test accuracy	71.74%	—
Test Area Under the Curve (AUC) (roc)	0.777	—
Early stopping	—	45

5.2. Test-Set Evaluation

Following training, the best-checkpoint model was evaluated on a held-out test set of 92 images. Table 5-2 presents the per-class precision, recall, and F1-score.

Table 5-2. Per-Class Classification Report on the Test Set (n = 92)

Class	**Precision**	**Recall**	**F1-score**	**Support**
Class 0 (No human)	0.6190	0.7222	0.6667	36
Class 1 (Human)	0.8000	0.7143	0.7547	56
Macro average	0.7095	0.7183	0.7107	92
Weighted average	0.7292	0.7174	0.7203	92
Overall test accuracy				**71.74%**

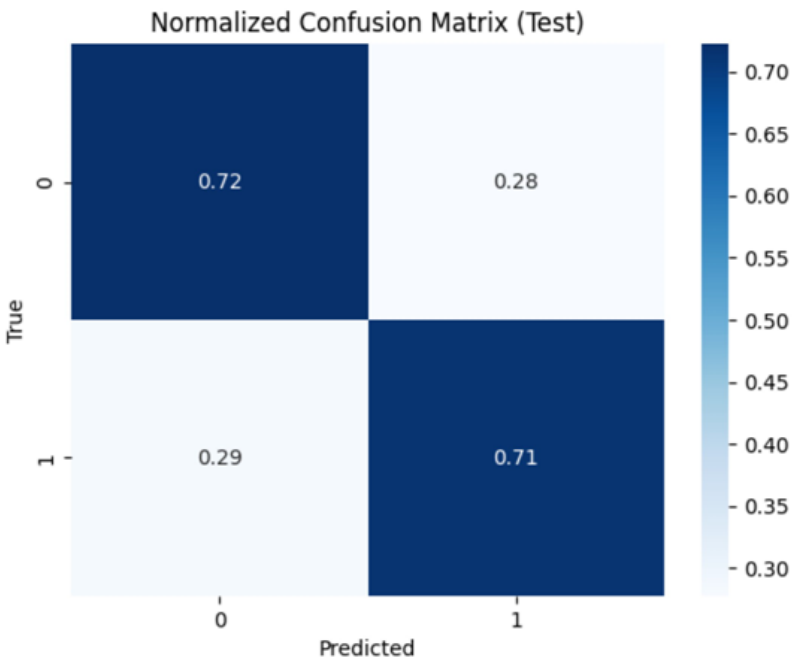

Figure 5-2. Normalised confusion matrix on the test set.

The model achieves a test accuracy of **71.74%** and an AUC of **0.777**. As shown in Fig. 5-2 and Table 5-2, the per-class recall is balanced (0.72 and 0.71), confirming that weighted sampling prevented a majority-class collapse. Notably, Class 1 precision (0.80) exceeds Class 0 precision (0.62), indicating the model favors human detection—a desirable property for surveillance applications where missed detections are more critical than false alarms.

6. REMAINING TASKS

CHAPTER GUIDE: Purpose: describe unfinished work, risks, and a realistic plan. Must include: What is completed vs pending, Remaining milestones, Risks/blockers + mitigation. LaTeX: Use itemize for checklists; enumerate for ordered milestones.

7. CONCLUSION

The Conclusion chapter summarizes your project, highlights key achievements, discusses limitations, and suggests future work. It should provide closure to your report and leave readers with a clear understanding of what you accomplished and its significance.

7.1. Achievements

7.2. Limitations

7.3. Future Work

7.4. Final Remarks

Highlight major accomplishments: Briefly recap the problem you addressed, your approach, and what you built. This should be 1-2 paragraphs summarizing the entire project without going into technical details.

- Whether objectives were met - Performance metrics achieved - Novel contributions or innovations - Successful integration or deployment Be specific and quantitative where possible.

Acknowledge limitations of your work honestly: - Technical constraints encountered - Performance limitations - Scope restrictions - Areas where further improvement is needed

FUTURE WORK: Suggest concrete directions for extending your project: - Additional features that could be implemented - Performance improvements that could be made - Alternative approaches that could be explored - Scaling or deployment in real-world settings - Research questions that emerged from your work

End with a brief statement about the significance of your work and its potential impact.

LaTeX quick refs: - Keep conclusion concise; avoid introducing new results. - If referencing key results: point to earlier figures/tables via

A. PROJECT BUDGET

Table A-1. Budget Estimation

S.N.	Hardware	Units	Cost (NPR)
1	ESP32-CAM	1	Rs. 1300
2	ESP32-S3	2	Rs. 3000
3	ESP32	1	Rs. 1000
4	FTDI	1	Rs. 350
5	Data Cables	4	Rs. 400
6	I2C LCD Display	1	Rs. 1000
7	Others	-	Rs. 1000
		Grand Total	**Rs. 8050**

B. PROJECT TIMELINE

B.1. Gantt Chart

Gantt Chart Placeholder here:

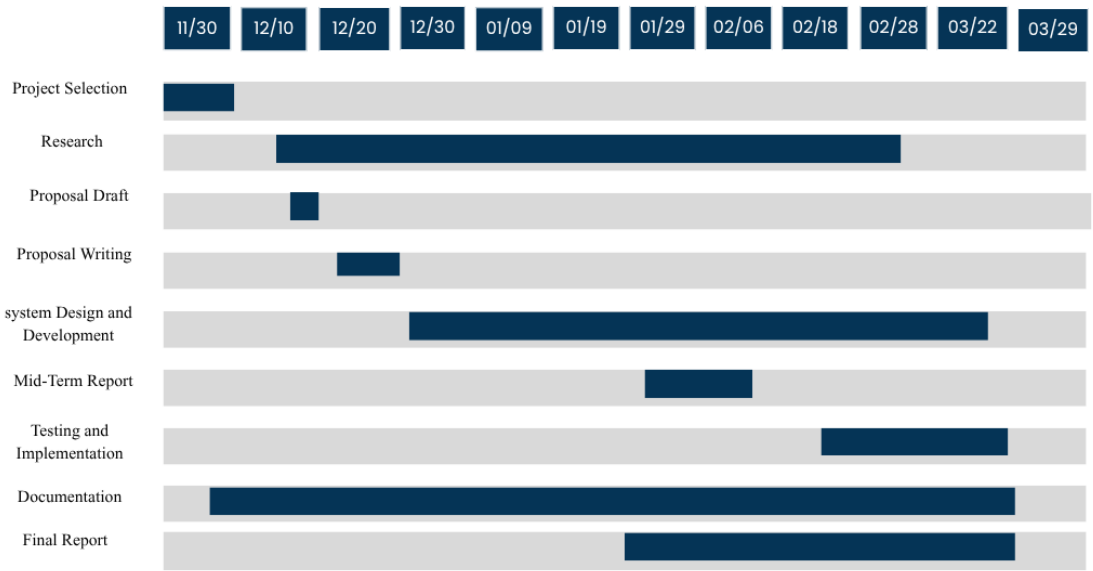

Table B-1. Gantt Chart for Project Timeline

C. IMPLEMEMNTED CODE

REFERENCES